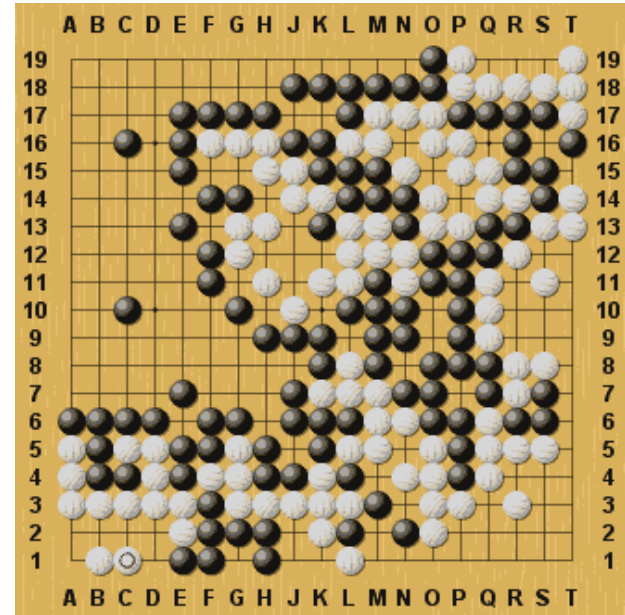


ML: Reinforcement learning

Game Playing

- Example: Go (one of the oldest and hardest board games)

- **Agent:** [redacted]
- **Environment:** [redacted]
- **State:** [redacted]
- **Action:** [redacted]
- **Reward:** [redacted]



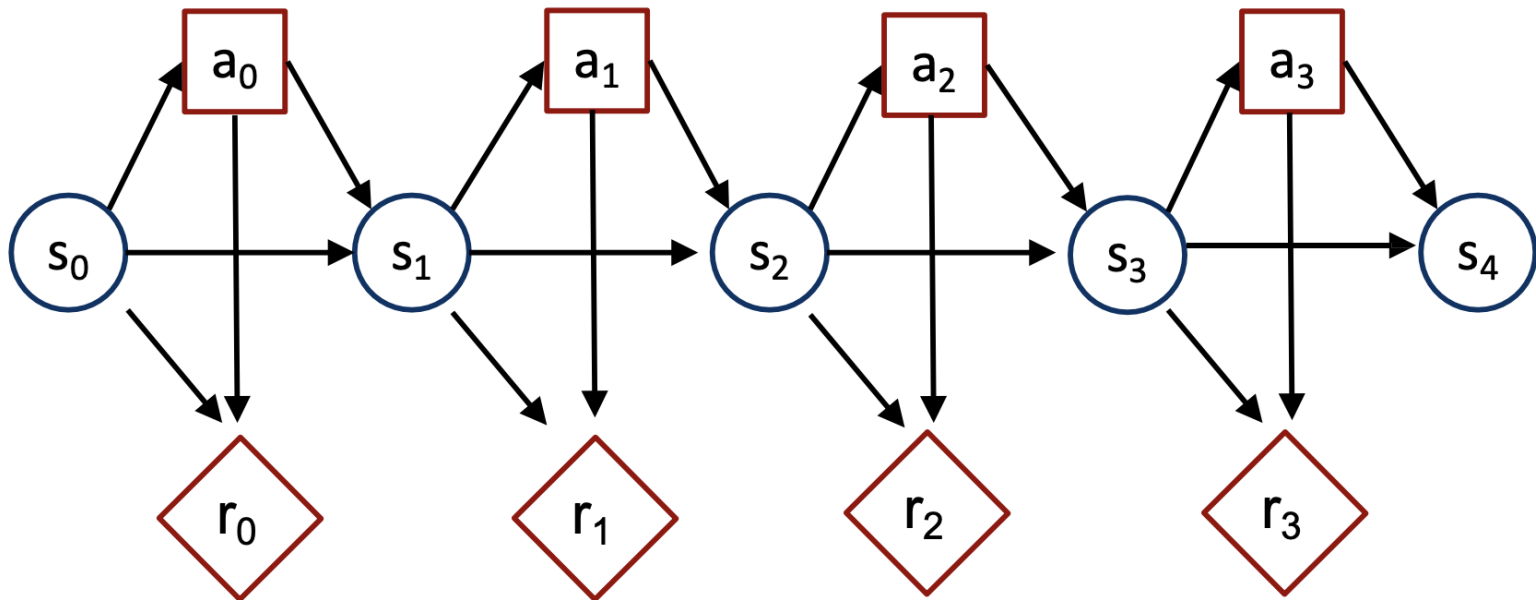
- 2016: AlphaGo defeats top player Lee Sedol (4-1)
 - Game 2 move 37: AlphaGo plays unexpected move (odds 1/10,000)

Markov Decision Process

- Definition
 - Set of states: S
 - Set of actions: A
 - Transition model: $\Pr(s_t | s_{t-1}, a_{t-1})$
 - Reward model: $R(s_t, a_t)$
 - Discount factor: $0 \leq \gamma \leq 1$
 - discounted: $\gamma < 1$ undiscounted: $\gamma = 1$
 - Horizon (i.e., # of time steps): h
 - Finite horizon: $h \in \mathbb{N}$ infinite horizon: $h = \infty$
- Goal: find optimal policy

Markov Decision Process

- Markov process augmented with...
 - Actions e.g., a_t
 - Rewards e.g., r_t



Current Assumptions

- Uncertainty: **stochastic** process
- Time: **sequential** process
- Observability: **fully** observable states (observe the environment completely and exactly)
- No learning: **complete** model
- Variable type: **discrete** (e.g., discrete states and actions)

Rewards

- **Rewards:** $r_t \in \mathbb{R}$
- **Reward function:** $R(s_t, a_t) = r_t$
mapping from state-action pairs to rewards
- Common assumption: **stationary** reward function
 - $R(s_t, a_t)$ is the same $\forall t$ (function)
- Exception: terminal reward function often different
 - E.g., in a game: 0 reward at each turn and +1/-1 at the end for winning/losing
- **Goal: maximize sum of rewards** $\sum_t R(s_t, a_t)$

Discounted/Average Rewards

- If process infinite, isn't $\sum_t R(s_t, a_t)$ infinite?
- Solution 1: **discounted rewards**
 - Discount factor: $0 \leq \gamma < 1$
 - Finite utility: $\sum_t \gamma^t R(s_t, a_t)$ is a geometric sum
 - γ induces an inflation rate of $1/\gamma - 1$
 - Intuition: prefer utility sooner than later
- Solution 2: **average rewards**
 - More complicated computationally
 - Beyond the scope of this course

Markov Decision Process

- Definition
 - Set of states: $\mathcal{S}$
 - Set of actions: $\mathcal{A}$
 - Transition model: $\Pr(s_t | s_{t-1}, a_{t-1})$
 - Reward model: $R(s_t, a_t)$
 - Discount factor: $0 \leq \gamma \leq 1$
 - discounted: $\gamma < 1$ undiscounted: $\gamma = 1$
 - Horizon (i.e., # of time steps): h
 - Finite horizon: $h \in \mathbb{N}$ infinite horizon: $h = \infty$
- Goal: find optimal policy

Policy

- Choice of action at each time step
- Formally:
 - Mapping from states to actions
 - i.e., $\pi(s_t) = a_t$
 - Assumption: **fully observable states**
 - Allows a_t to be chosen only based on current state s_t

Policy Optimization

- Policy evaluation:
 - Compute expected utility

$$V^{\pi}(s_0) = \sum_{t=0}^h \gamma^t \sum_{s_t} \Pr(s_t | s_0, \pi) R(s_t, \pi(s_t))$$

- Optimal policy:
 - Policy with highest expected utility

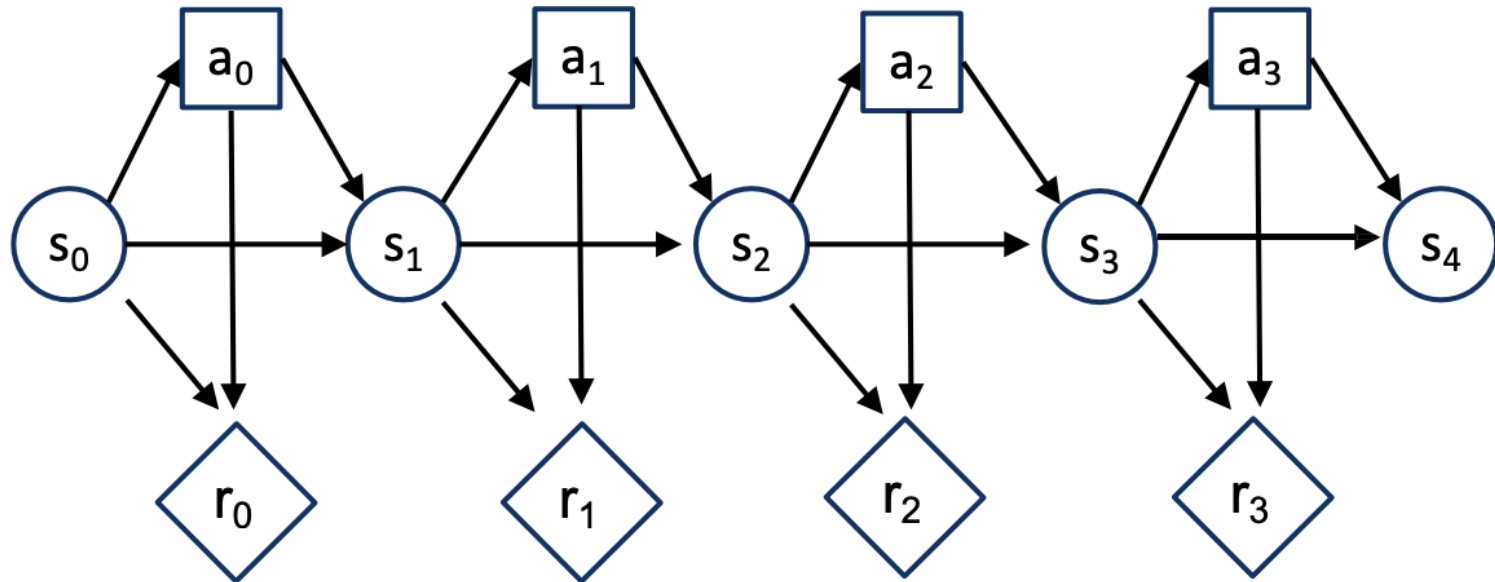
$$V^{\pi^*}(s_0) \geq V^{\pi}(s_0) \quad \forall \pi$$

Policy Optimization

- Several classes of algorithms:
 - Value iteration
 - Policy iteration
 - Linear Programming
 - Search techniques
- Computation may be done
 - Offline: before the process starts
 - Online: as the process evolves

Value Iteration

- Performs dynamic programming
- Optimizes decisions in reverse order



Value Iteration

- Value when no time left:

$$V(s_h) = \max_{a_h} R(s_h, a_h)$$

- Value with one time step left:

$$V(s_{h-1}) = \max_{a_{h-1}} R(s_{h-1}, a_{h-1}) + \gamma \sum_{s_h} \Pr(s_h | s_{h-1}, a_{h-1}) V(s_h)$$

- Value with two time steps left:

$$V(s_{h-2}) = \max_{a_{h-2}} R(s_{h-2}, a_{h-2}) + \gamma \sum_{s_{h-1}} \Pr(s_{h-1} | s_{h-2}, a_{h-2}) V(s_{h-1})$$

- ...

- **Bellman's equation:**

$$V(s_t) = \max_{a_t} R(s_t, a_t) + \gamma \sum_{s_{t+1}} \Pr(s_{t+1} | s_t, a_t) V(s_{t+1})$$

$$a_t^* = \underset{a_t}{\operatorname{argmax}} R(s_t, a_t) + \gamma \sum_{s_{t+1}} \Pr(s_{t+1} | s_t, a_t) V(s_{t+1})$$

Value Iteration Algorithm

valueIteration(MDP)

$$V_0^*(s) \leftarrow \max_a R(s, a) \quad \forall s$$

For $t = 1$ to h do

$$V_t^*(s) \leftarrow \max_a R(s, a) + \gamma \sum_{s'} \Pr(s'|s, a) V_{t-1}^*(s') \quad \forall s$$

Return V^*

Optimal policy π^*

$$t = 0: \pi_0^*(s) \leftarrow \arg\max_a R(s, a) \quad \forall s$$

$$t > 0: \pi_t^*(s) \leftarrow \arg\max_a R(s, a) + \gamma \sum_{s'} \Pr(s'|s, a) V_{t-1}^*(s') \quad \forall s$$

NB: t indicates the # of time steps to go (till end of process)

π^* is **non stationary** (i.e., time dependent)

Value Iteration

- Matrix form:

R^a : $|S| \times 1$ column vector of rewards for a

V_t^* : $|S| \times 1$ column vector of state values

T^a : $|S| \times |S|$ matrix of transition prob. for a

valueiteration(MDP)

$$V_0^* \leftarrow \max_a R^a$$

For $t = 1$ to h do

$$V_t^* \leftarrow \max_a R^a + \gamma T^a V_{t-1}^*$$

Return V^*

Policy Optimization

- Value iteration
 - Optimize value function
 - Extract induced policy
- Can we directly optimize the policy?
 - Yes, by **policy iteration**

Policy Iteration

- Alternate between two steps

1. Policy evaluation

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} \Pr(s'|s, \pi(s)) V^\pi(s') \quad \forall s$$

2. Policy improvement

$$\pi(s) \leftarrow \operatorname{argmax}_a R(s, a) + \gamma \sum_{s'} \Pr(s'|s, a) V^\pi(s') \quad \forall s$$

Algorithm

policyIteration(MDP)

Initialize π_0 to any policy

$n \leftarrow 0$

Repeat

 Eval: $V_n = R^{\pi_n} + \gamma T^{\pi_n} V_n$

 Improve: $\pi_{n+1} \leftarrow \operatorname{argmax}_a R^a + \gamma T^a V_n$

$n \leftarrow n + 1$

Until $\pi_{n+1} = \pi_n$

Return π_n

Policy evaluation

- Linear system of equations

$$V_{\infty}^{\pi}(s) = R(s, \pi_{\infty}(s)) + \gamma \sum_{s'} \Pr(s'|s, \pi_{\infty}(s)) V_{\infty}^{\pi}(s') \quad \forall s$$

- Matrix form:

$R: |S| \times 1$ column vector of state rewards for π

$V: |S| \times 1$ column vector of state values for π

$T: |S| \times |S|$ matrix of transition prob for π

$$V = R + \gamma TV$$

Handwritten notes illustrating policy evaluation for a two-state MDP:

- States: s_1, s_2
- Policy: $\pi(s_1) = a_1, \pi(s_2) = a_2 \rightarrow$ given fixed policy
- Reward vector: $R^{\pi} = \begin{pmatrix} 0 \\ 5 \end{pmatrix}$
- Transition matrix: $T^{\pi} = \begin{pmatrix} 0.3 & 0.7 \\ 0.2 & 0.8 \end{pmatrix}$
- Transitions: $s_1 \rightarrow a_1$ (0.3 to s_1 , 0.7 to s_2), $s_2 \rightarrow a_2$ (0.2 to s_1 , 0.8 to s_2)

Reinforcement Learning

- Definition
 - States: $s \in S$
 - Actions: $a \in A$
 - Rewards: $r \in \mathbb{R}$
 - Transition model: $\Pr(s_t | s_{t-1}, a_{t-1})$
 - Reward model: $\Pr(r_t | s_t, a_t)$
 - Discount factor: $0 \leq \gamma \leq 1$
 - discounted: $\gamma < 1$ undiscounted: $\gamma = 1$
 - Horizon (i.e., # of time steps): h
 - Finite horizon: $h \in \mathbb{N}$ infinite horizon: $h = \infty$
- Goal: **find optimal policy π^*** such that
$$\pi^* = \operatorname{argmax}_{\pi} \sum_{t=0}^h \gamma^t E_{\pi}[r_t]$$

Policy optimization

- Markov Decision Process:
 - Find optimal policy given transition and reward model
 - Execute policy found
- Reinforcement learning:
 - Learn an optimal policy while interacting with the environment

Important Components in RL

RL agents may or may not include the following components:

- **Model**: $\Pr(s' | s, a), \Pr(r | s, a)$
 - Environment dynamics and rewards
- **Policy**: $\pi(s)$
 - Agent action choices
- **Value function**: $V(s)$
 - Expected total rewards of the agent policy

Categorizing RL agents

Value based

- No policy (implicit)
- Value function

Policy based

- Policy
- No value function

Actor critic

- Policy
- Value function

Model based

- Transition and reward model

Model free

- No transition and no reward model (implicit)

Model free evaluation

- Given a policy π , estimate $V^\pi(s)$ without any transition or reward model

- **Monte Carlo** evaluation

$$\begin{aligned} V^\pi(s) &= E_\pi \left[\sum_t \gamma^t r_t \right] \\ &\approx \frac{1}{n(s)} \sum_{k=1}^{n(s)} [\sum_t \gamma^t r_t^{(k)}] \quad (\text{sample approximation}) \end{aligned}$$

- **Temporal difference (TD)** evaluation

$$\begin{aligned} V^\pi(s) &= E[r|s, \pi(s)] + \gamma \sum_{s'} \Pr(s'|s, \pi(s)) V^\pi(s') \\ &\approx r + \gamma V^\pi(s') \quad (\text{one sample approximation}) \end{aligned}$$

Monte Carlo Evaluation

- Let G_k be a one-trajectory Monte Carlo target

$$G_k = \sum_t \gamma^t r_t^{(k)}$$

- Approximate value function

$$\begin{aligned} V_n^\pi(s) &\approx \frac{1}{n(s)} \sum_{k=1}^{n(s)} G_k \\ &= \frac{1}{n(s)} (G_{n(s)} + \sum_{k=1}^{n(s)-1} G_k) \\ &= \frac{1}{n(s)} (G_{n(s)} + (n(s) - 1)V_{n-1}^\pi(s)) \\ &= V_{n-1}^\pi(s) + \frac{1}{n(s)} (G_{n(s)} - V_{n-1}^\pi(s)) \end{aligned}$$

- Incremental update**

$$V_n^\pi(s) \leftarrow V_{n-1}^\pi(s) + \alpha_n (G_{n(s)} - V_{n-1}^\pi(s))$$

learning rate $1/n(s)$

Temporal Difference Evaluation

- Approximate value function: $V^\pi(s) \approx r + \gamma V^\pi(s')$
- **Incremental update**

$$V_n^\pi(s) \leftarrow V_{n-1}^\pi(s) + \alpha_n (r + \gamma V_{n-1}^\pi(s') - V_{n-1}^\pi(s))$$

- **Theorem:** If α_n is appropriately decreased with number of times a state is visited then $V_n^\pi(s)$ converges to correct value
- **Sufficient conditions** for α_n :
 - (1) $\sum_n \alpha_n \rightarrow \infty$
 - (2) $\sum_n (\alpha_n)^2 < \infty$
- Often $\alpha_n = 1/n(s)$
 - Where $n(s)$ = # of times s is visited

Temporal Difference (TD) evaluation

TDevaluation(π, V^π)

Repeat

Execute $\pi(s)$

Observe s' and r

Update counts: $n(s) \leftarrow n(s) + 1$

Learning rate: $\alpha \leftarrow 1/n(s)$

Update value: $V^\pi(s) \leftarrow V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s))$

$s \leftarrow s'$

Until convergence of V^π

Return V^π

Model Free Control

- Instead of evaluating the state value fn, $V^\pi(s)$, evaluate the state-action value fn, $Q^\pi(s, a)$

$Q^\pi(s, a)$: value of executing a followed by π

$$Q^\pi(s, a) = E[r|s, a] + \gamma \sum_{s'} \Pr(s'|s, a) V^\pi(s')$$

- Greedy policy π'

$$\pi'(s) = \operatorname{argmax}_a Q^\pi(s, a)$$

Bellman's Equation

- Optimal state value function $V^*(s)$

$$V^*(s) = \max_a E[r|s, a] + \gamma \sum_{s'} \Pr(s'|s, a) V^*(s')$$

- Optimal state-action value function $Q^*(s, a)$

$$Q^*(s, a) = E[r|s, a] + \gamma \sum_{s'} \Pr(s'|s, a) \max_{a'} Q^*(s', a')$$

where $V^*(s) = \max_a Q^*(s, a)$

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Monte Carlo Control

- Let G_k^a be a one-trajectory Monte Carlo target

$$G_k^a = \underbrace{r_0^{(k)}}_a + \underbrace{\sum_{t=1} \gamma^t r_t^{(k)}}_\pi$$

- Alternate between
 - **Policy evaluation**

$$Q_n^\pi(s, a) \leftarrow Q_{n-1}^\pi(s, a) + \alpha_n (G_n^a - Q_{n-1}^\pi(s, a))$$

- **Policy improvement**

$$\pi'(s) \leftarrow \operatorname{argmax}_a Q^\pi(s, a)$$

Q-Learning

Qlearning(s, Q^*)

Repeat

Select and execute a

Observe s' and r

Update counts: $n(s, a) \leftarrow n(s, a) + 1$

Learning rate: $\alpha \leftarrow 1/n(s, a)$

Update Q-value:

$$Q^*(s, a) \leftarrow Q^*(s, a) + \alpha(r + \gamma \max_{a'} Q^*(s', a') - Q^*(s, a))$$

$$s \leftarrow s'$$

Until convergence of Q^*

Return Q^*

| **S** | 太大怎么办？

Q-function Approximation

- Let $s = (x_1, x_2, \dots, x_n)^T$

- Linear

$$Q(s, a) \approx \sum_i w_{ai} x_i$$

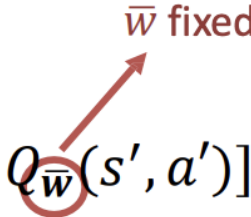
- Non-linear (e.g., neural network)

$$Q(s, a) \approx g(\mathbf{x}; \mathbf{w})$$

Gradient Q-learning

- Minimize squared error between Q-value estimate and target
 - Q-value estimate: $Q_{\mathbf{w}}(s, a)$
 - Target: $r + \gamma \max_{a'} Q_{\bar{\mathbf{w}}}(s', a')$

- Squared error:

$$Err(\mathbf{w}) = \frac{1}{2} [Q_{\mathbf{w}}(s, a) - r - \gamma \max_{a'} Q_{\bar{\mathbf{w}}}(s', a')]^2$$


- Gradient

$$\frac{\partial Err}{\partial \mathbf{w}} = [Q_{\mathbf{w}}(s, a) - r - \gamma \max_{a'} Q_{\bar{\mathbf{w}}}(s', a')] \frac{\partial Q_{\mathbf{w}}(s, a)}{\partial \mathbf{w}}$$

Gradient Q-learning

Initialize weights w uniformly at random in $[-1,1]$

Observe current state s

Loop

 Select action a and execute it

 Receive immediate reward r

 Observe new state s'

 Gradient: $\frac{\partial Err}{\partial \mathbf{w}} = [Q_{\mathbf{w}}(s, a) - r - \gamma \max_{a'} Q_{\mathbf{w}}(s', a')] \frac{\partial Q_{\mathbf{w}}(s, a)}{\partial \mathbf{w}}$

 Update weights: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial Err}{\partial \mathbf{w}}$

 Update state: $s \leftarrow s'$

Convergence of Linear Gradient Q-Learning

- Linear Q-Learning converges under the same conditions:

$$\sum_{n=0}^{\infty} \alpha_n = \infty \text{ and } \sum_{n=0}^{\infty} \alpha_n^2 < \infty$$

- Let $\alpha_n = 1/n$
- Let $Q_w(s, a) = \sum_i w_i x_i$
- Q-learning

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha_n [Q_{\mathbf{w}}(s, a) - r - \gamma \max_{a'} Q_{\mathbf{w}}(s', a')] \frac{\partial Q_{\mathbf{w}}(s, a)}{\partial \mathbf{w}}$$

Mitigating divergence

- Two simple tricks are often used in practice:
 1. Experience replay
 - Advantages
 - Break correlations between successive updates (more stable learning)
 - Fewer interactions with environment needed to converge (greater data efficiency)
 2. Use two networks:
 - Q-network
 - Target network

Deep Q-network

Initialize weights w and $\bar{w}$ random in $[-1,1]$

Observe current state s

Loop

 Select action a and execute it

 Receive immediate reward r

 Observe new state s'

 Add (s, a, s', r) to experience buffer

 Sample mini-batch of experiences from buffer

 For each experience $(\hat{s}, \hat{a}, \hat{s}', \hat{r})$ in mini-batch

$$\text{Gradient: } \frac{\partial \text{Err}}{\partial w} = [Q_w(\hat{s}, \hat{a}) - \hat{r} - \gamma \max_{a'} Q_{\bar{w}}(\hat{s}', \hat{a}')] \frac{\partial Q_w(\hat{s}, \hat{a})}{\partial w}$$

$$\text{Update weights: } w \leftarrow w - \alpha \frac{\partial \text{Err}}{\partial w}$$

Update state: $s \leftarrow s'$

Every c steps, update target: $\bar{w} \leftarrow w$

Deep Q-Network for Atari

